CprE 3810, Computer Organization and Assembly Level Programming

Lab 1 Report

Student Name Shawn Robinson

Submit a typeset pdf version of this on Canvas by the due date. Refer to the highlighted language in the lab document for the context of the following questions.

[Part 3.1.c] Think of three more cases and record them in your lab report.

Shawn's Test Case #1 – Sum up to 7 (the day I was born, and also the month for what it's worth)
-Let's set iX to 03, a weight of 1, iLdw to 1, and iY to 04
-with a multiplied weight of 1, this should create the sum of 7

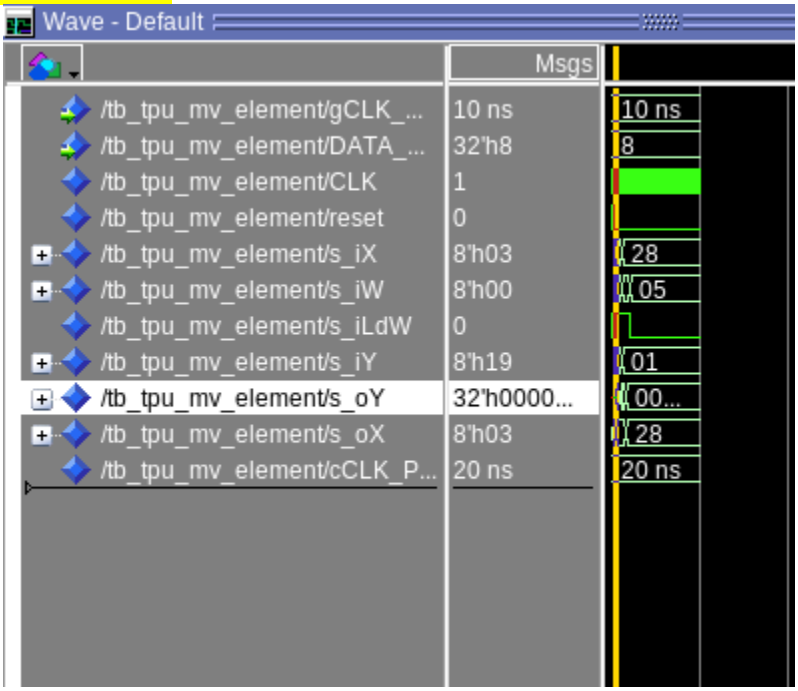
Shawn's Test Case #2 – Sum up to F1, like the racing sports league!
-F1 is 16*15 + 1 = 241...so, let's load 6 as the weight, 40 as iX and 1 as iY

Shawn's Test Case #3 – Testing to assert that weights don't change when iLdw is zero
-Keep the weight at 6 if iLdw is 0, even if a new weight value is given

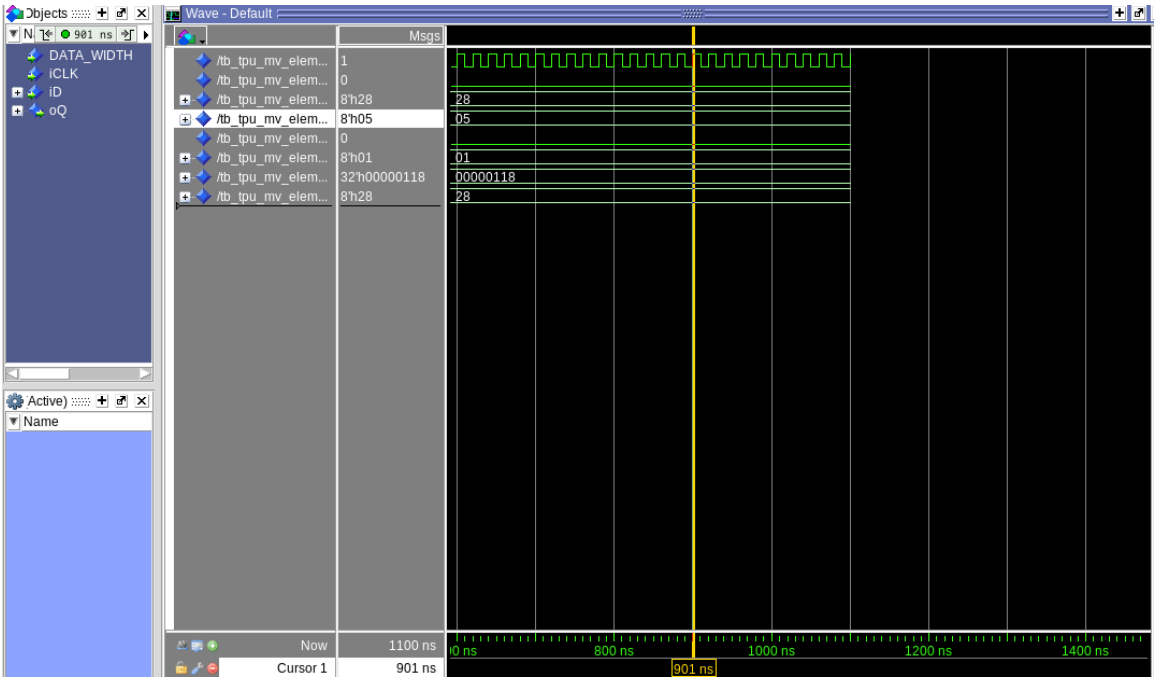
[Part 3.1.e] For labels 1, 7, 22, and 28, specify where (VHDL file and line number) these values are located – some will be found in more than one place. Also attempt to explain the functionality of each label as it occurs in the code

Label 1: TPU_MV_Element
Location: TPU_MV_Element.vhd – line number 24 is where the entity is defined
Description: This is the top-level of the hierarchy of components for this particular design, it contains and declares what other sub-modules it needs to function as a whole
Label 7: g_Add1
Location: TPU_MV_Element.vhd – line number 131
Description: This component takes the multiplication of iX * iW and adds it with iY, and this sum then flows onto oY after passing through a clock entity (a delay, or maybe just shorthand to say that that result will push onward with the next rising edge?)
Label 22: oQ
Location: RegLd.vhd – line number 30
Description: oQ acts as both the output for what gets sent as the weight to be multiplied later on, but also gets sent back through the 2:1 mux, so if iLdw is set to 0, then the value of oQ remains constant. However, if a new value comes in from iD and iLd is 1, then oQ is rewritten.
Label 28: s_X1
Location: TPU_MV_Element.vhd – line number 70
Description: I believe this is an intermediate signal which is carrying the value of oQ from g_Delay1 to iD of g_Delay3

[Part 3.1.f] Implement your test cases from part c into this testbench. Remember to include these in your waveform screenshot.



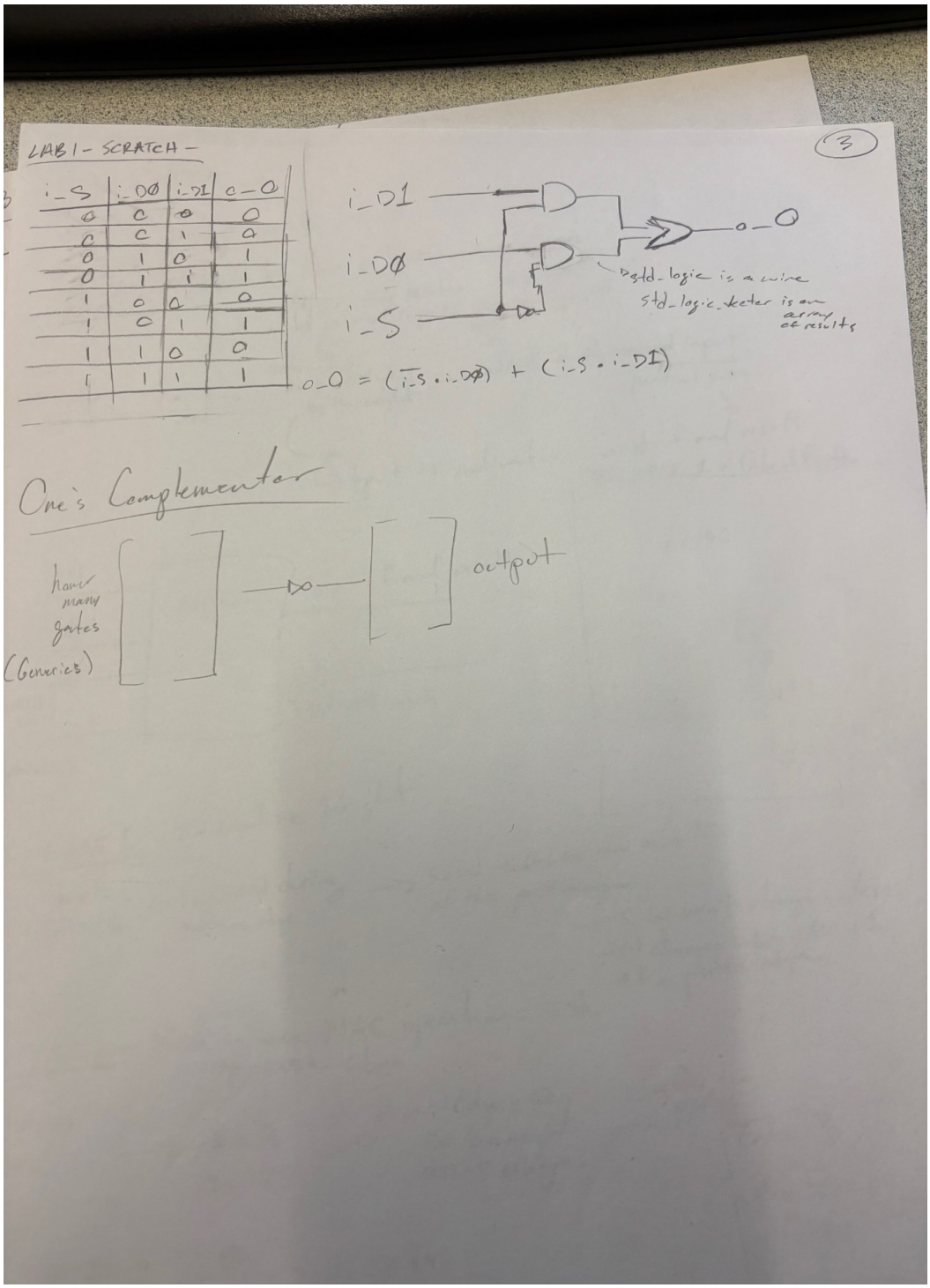
[Part 3.1.g.v] In your lab report, include a screenshot of the waveform. Describe, in plain English, any differences between what you expected and what the simulation showed.



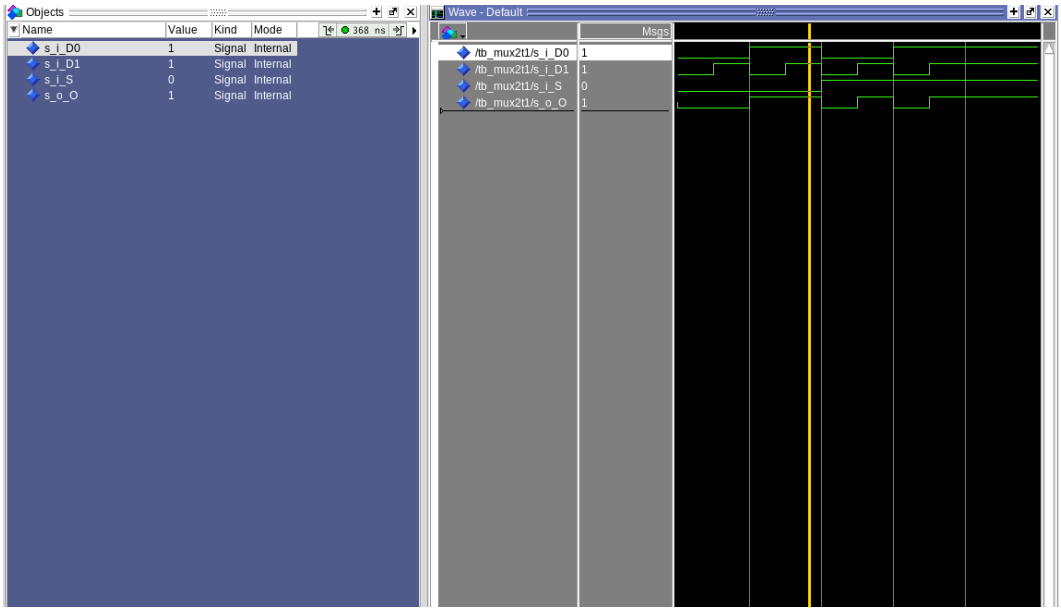
Fixes:
g_Delay2 needed to be changed...iD gets assigned to iY

[Part 3.1.h] In your lab report, include a screenshot of the waveform. Describe, in plain English, how your waveform matches the expected result (e.g., reference the specific cycles and times). In your submission zip file, provide the completed *TPU_MV_Element.vhd* file in a folder called 'MAC'.

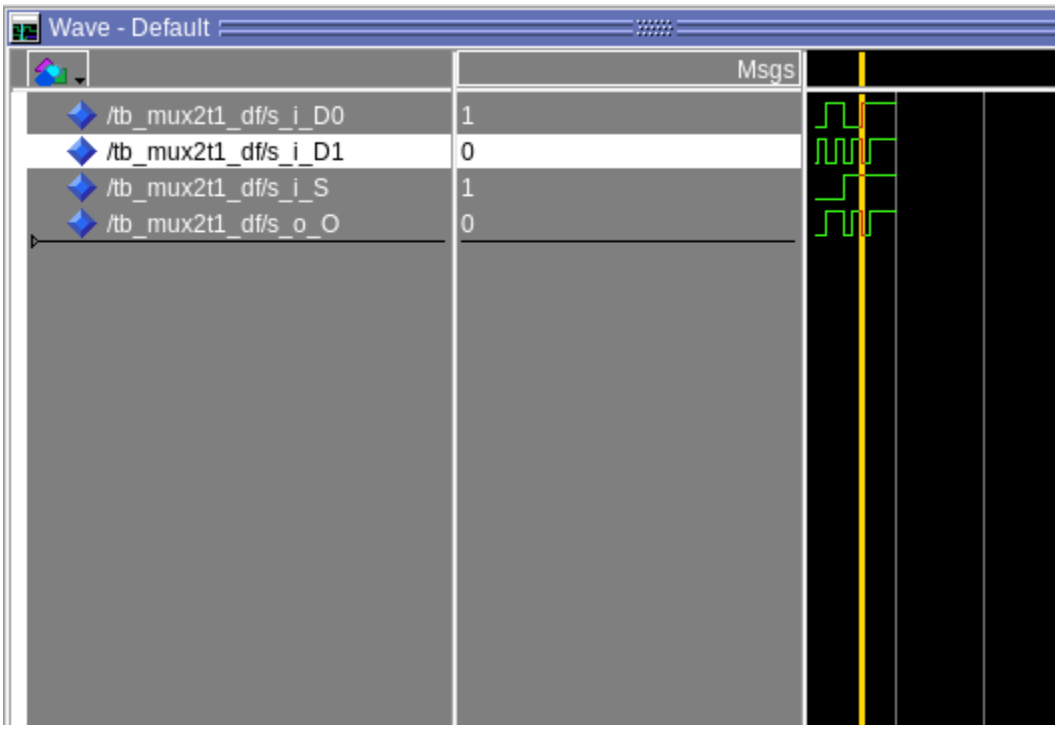
[Part 3.3.a] Draw the truth table, Boolean equation, and Boolean circuit equivalent (using only two-input gates) that implements a 2:1 mux. Include this in your lab report.



[Part 3.3.d] In your lab report, include a screenshot of the waveform. Make sure to label the screenshot with which module it is testing.

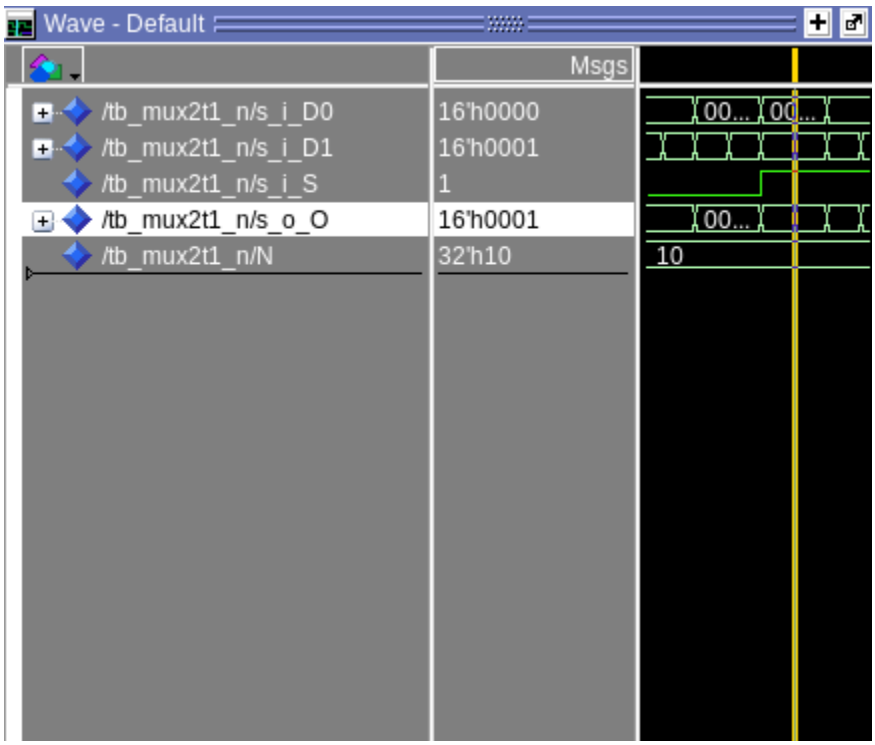


[Part 3.3.e] Again, in your lab report, include a labeled screenshot of the waveform showing the dataflow mux implementation working.



[Part 3.4] Include a waveform screenshot and corresponding description demonstrating it is working correctly.

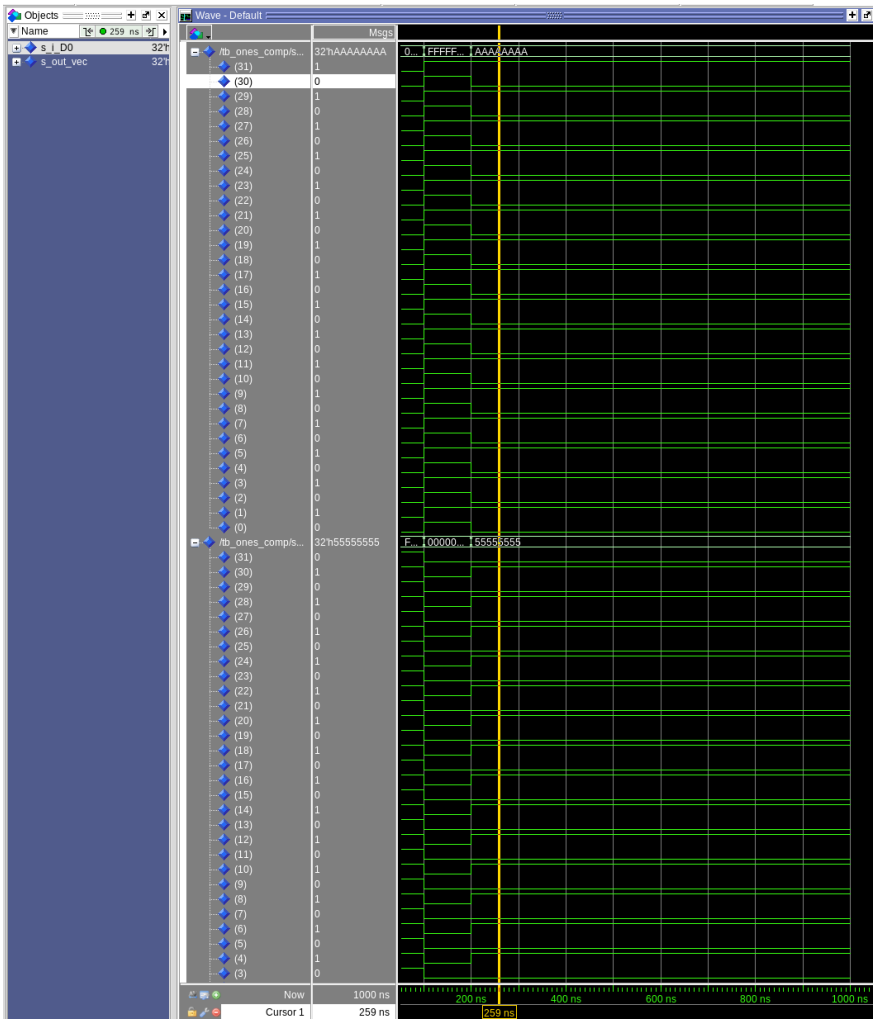
Description: here you can see the testbench I wrote working successfully. The screenshot in question shows a situation where the select is set to 1, and d1 is also 1 – thus, the output is 1.



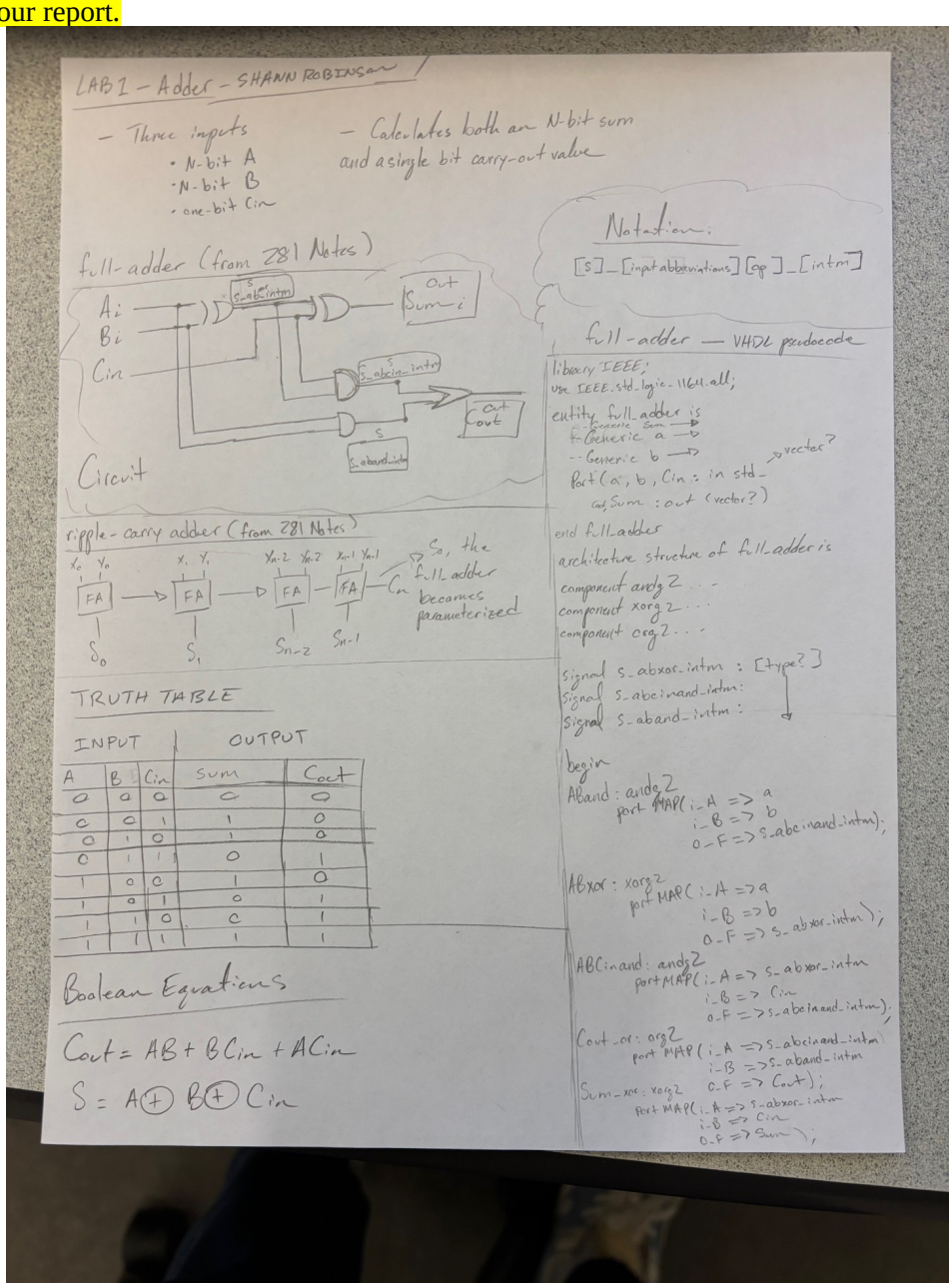
[Part 3.5.b] Include a waveform screenshot and description in your lab report.

Description: My testbench used all zeroes, all 1's, and all A's

I got the expected outputs of F's, zeroes, and fives – respectively.



[Part 3.6.a] A full adder takes three single-bit inputs and produces two single-bit outputs – a sum and carry for the addition of the three input bits. Draw the truth table, Boolean equation, and Boolean circuit equivalent (using only two-input gates) that implements a 1-bit full adder. Include this in your report.



[Part 3.6.c] Then draw a schematic of the intended design, including inputs and outputs and at least the 0, 1, N-2, and N-1 stages. Include this in your report.

[My above screenshot also includes the schematic]

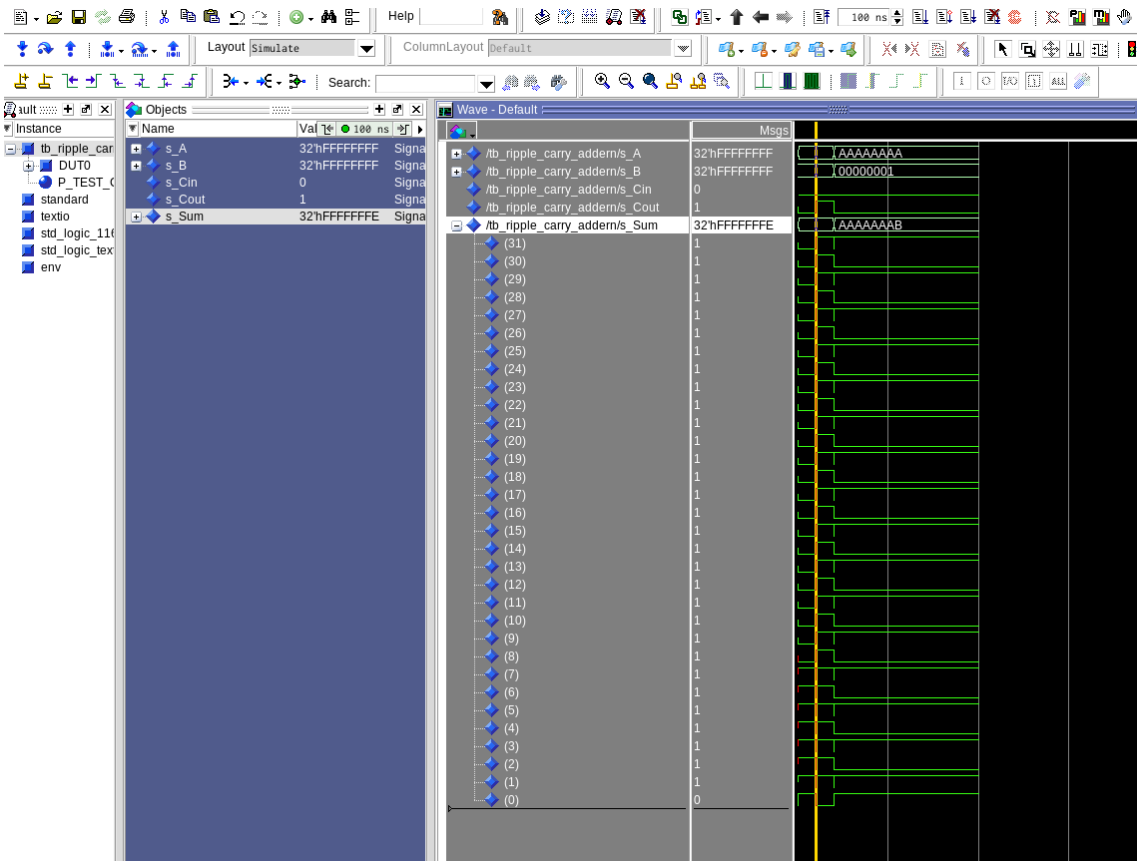
THREE INPUTS AND EXPECTED OUTPUTS:

1) 0000_0000_1111_1111 + 0000_0000_0000_0000 => 0000_0000_1111_1111 with carry_out 0

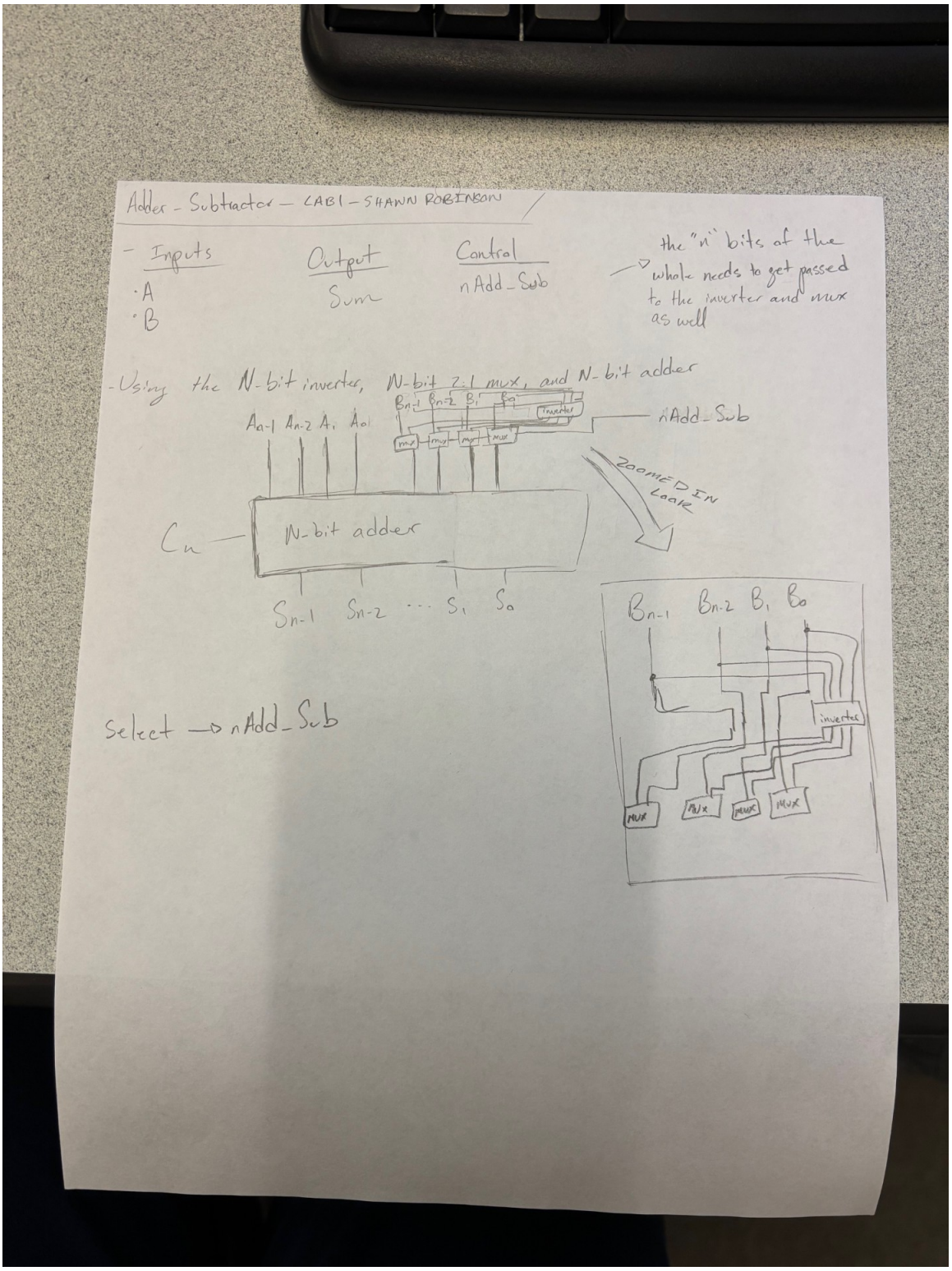
2) 1111_1111_1111_1111 + 1111_1111_1111_1111 => 1111_1111_1111_1110 with carry_out 1

3) 1010_1010_1010_1010 + 0000_0000_0000_0001 = > 1010_1010_1010_1011 with carry_out 0

[Part 3.6.d] Include an annotated waveform screenshot in your write-up.

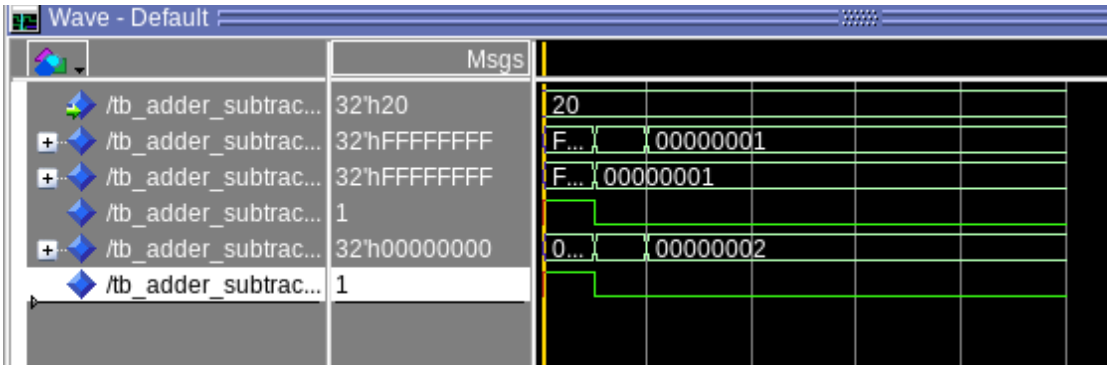


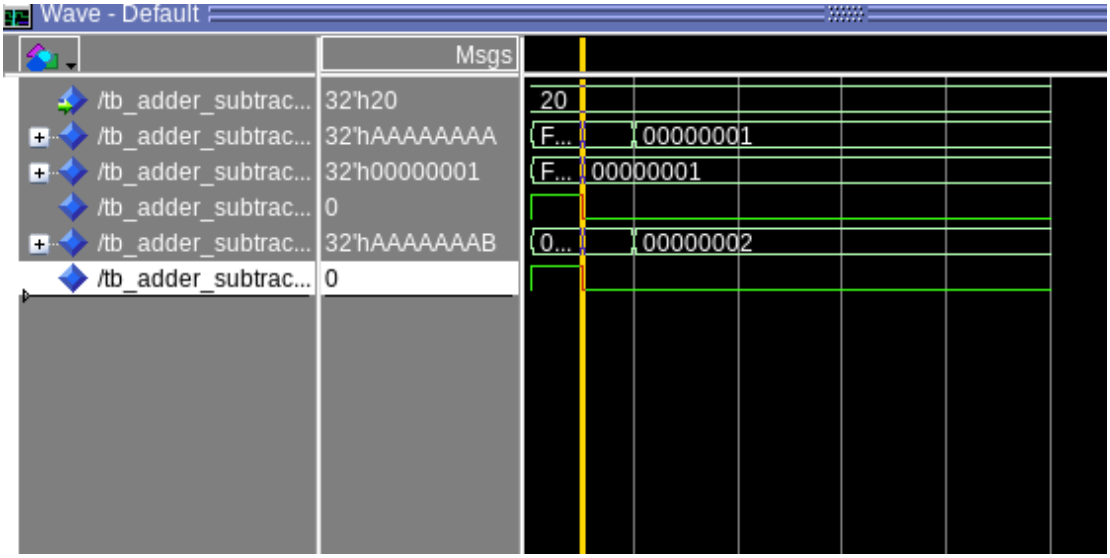
[Part 3.7.a] Draw a schematic (don't use a schematic capture tool) showing how an N-bit adder/subtractor with control can be implemented using <u>only</u> the three main components designed in earlier parts of this lab (i.e., the N-bit inverter, N-bit 2:1 mux, and N-bit adder). How is the 'nAdd_Sub' bit used? Include this in your report

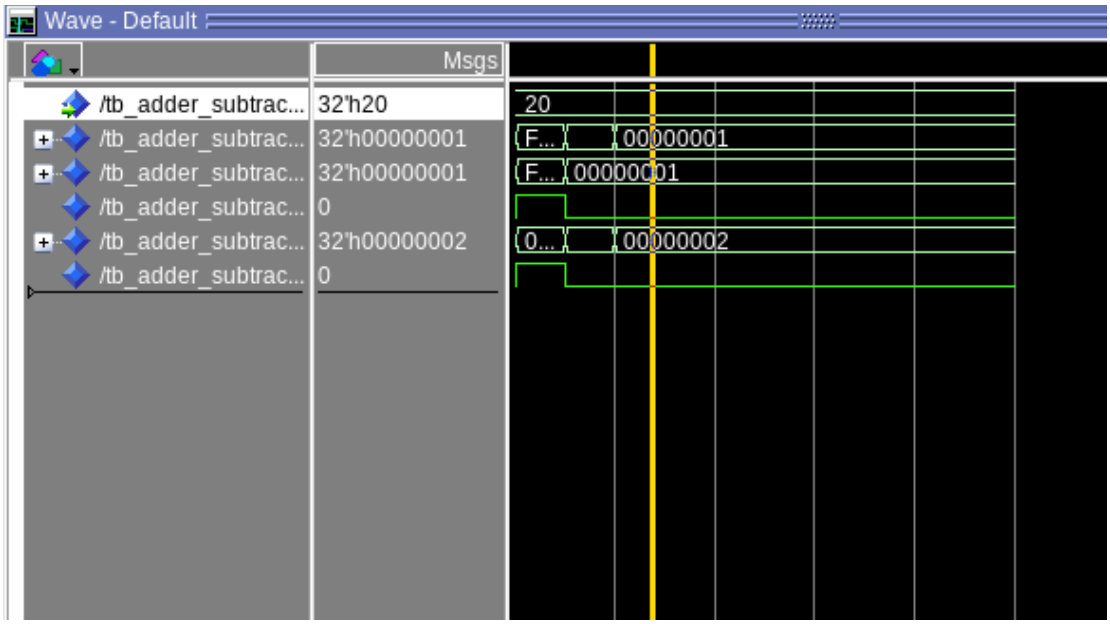


[Part 3.7.c] Provide multiple waveform screenshots in your write-up to confirm that this component is working correctly. What test-cases did you include and why?

I tested both addition and subtraction test cases, and tried to toggle whether there should be a carry out or not as part of the test. I know this isn't currently the most extensive set of tests, but I wanted them to be fairly comprehensive for their quantity.






Wave - Default
Msgs
/tb_adder_subtrac...
32'h20
/tb_adder_subtrac...
32'h00000001
/tb_adder_subtrac...
32'h00000001
/tb_adder_subtrac...
0
/tb_adder_subtrac...
32'h00000002
/tb_adder_subtrac...
0
20
F...
00000001
F...
00000001
0...
00000002